

MOBAN Chapter 4 — Comprehensive Study Guide

Fundamental Network Modelling Techniques: Optimisation

PART 1: THEORY SUMMARY

1. Introduction & General Concepts

Three phases for solving telecom network problems:

1. **Mathematical modelling** — translate the realistic problem into a model (graph, MILP, ...)
2. **Solving** — use a software solver (exactly or approximately)
3. **Interpretation** — is the solution realistic? Sensitivity? Comparison with alternatives?

Key terminology (from the optimisation landscape diagram):

Term	Definition
Objective function $f(x)$	Scalar-valued function to minimise/maximise
Decision variable x	Can be continuous (real) or discrete (integer/boolean)
Constraints	Equality/inequality functions defining the feasible region
Solution space X	All feasible values of x (satisfying all constraints)
Neighbourhood $N(x)$	Set of solutions $x' \in X$ adjacent to x (via a defined "move")
Move / trajectory	A single step from x to x' ; successive moves define a trajectory
Local optimum x_{lo}	At least as good as all points in $N(x_{lo})$
Global optimum x_{go}	At least as good as ALL points in X

General form of an optimisation problem:

```

minimise or maximise:  $c(x)$ 
subject to:
 $a_i(x) = 0$             $i = 1, \dots, m_1$ 
 $a_i(x) \leq 0$          $i = m_1+1, \dots, m_1+m_2$ 
 $a_i(x) \geq 0$          $i = m_1+m_2+1, \dots, m$ 

```

- **LP / heuristic split:** When $c(\cdot)$ and $a_i(\cdot)$ are all **linear** → use Linear Programming. Otherwise (or too complex) → use **heuristics**.

2. Linear Programming (LP) Techniques

2.1 The LP Problem

General form:

```

minimise or maximise:  $z = (c_0 +) c_1x_1 + c_2x_2 + \dots + c_nx_n$ 
subject to: functional constraints ( $=, \leq, \geq$ )
            sign restrictions:  $x_i \geq 0, x_i \leq 0, \text{ or } x_i \text{ unres}$ 

```

Four LP assumptions:

- **Proportionality** — each variable's contribution is proportional to its value (no economies of scale)
- **Additivity** — total function = sum of individual contributions (no interaction between variables)
- **Divisibility** — variables can take fractional values
- **Certainty** — all parameters are known constants

Four problem types (based on variable types):

Type	Variables	Complexity
LP	All real-valued	Relatively easy (polynomial via Karmarkar; fast in practice via Simplex)
ILP / IP	All integer	Much harder (NP-complete)
MIP / MILP	Some integer, some real	Much harder
BIP / BILP	All boolean (0/1)	Much harder

2.2 Standard Form & Transformation

Standard form (what Simplex needs):

```

maximise:  $z = c_1x_1 + c_2x_2 + \dots + c_nx_n$ 
subject to: equality constraints ( $=$ ) and  $\leq$  constraints
             $x_i \geq 0$  for all  $i$ 

```

Four transformation steps (general → standard form):

Step	Situation	Action
1	Minimisation problem	$z' = -z$ (now maximise)
2	$\geq$ constraint	Multiply by $-1 \rightarrow$ becomes $\leq$
3	$x_i \leq 0$ (nonpositive)	Replace with $x_i' = -x_i \geq 0$

Step	Situation	Action
4	x_i urs (unrestricted)	Replace with $x_i = x_i' - x_i''$ where $x_i', x_i'' \geq 0$

2.3 The Simplex Algorithm

Key insight: For an LP problem, the optimal solution (if it exists) is always at a **corner point** (basic feasible solution, BFS) of the feasible region. The Simplex walks from one BFS to an adjacent better BFS.

Translating to Simplex form — adding slack variables:

Each $\leq$ constraint $a_1x_1 + \dots + a_nx_n \leq b$ becomes an equality by adding a slack variable $s \geq 0$:

$$a_1x_1 + \dots + a_nx_n + s = b$$

Basic/non-basic variables:

- n variables, m functional constraints $\rightarrow$ set $n-m$ variables to 0 (= **nonbasic variables**)
- The remaining m variables are **basic variables** (solved from equality constraints)
- A basic solution where all basic variables ≥ 0 is a **basic feasible solution (BFS)**
- **Every corner point is a BFS, and vice versa**

Simplex algorithm — general description:

Initialisation:

1. Put LP into standard form S
2. Add slack variables $\rightarrow$ translated form T
3. Find a starting BFS (readily apparent if all constraints are $\leq$ with positive RHS; otherwise use Big M or Two-Phase method)

Iteration (repeat until optimal):

1. **Transformation:** Express z and basic variables as functions of nonbasic variables (row operations)
2. **Optimality test:** If all nonbasic variables have coefficients ≤ 0 in the z -expression $\rightarrow$ current BFS is optimal
3. **Basis change:**
 - **Entering variable:** nonbasic variable with the **largest positive coefficient** in z
 - **Leaving variable:** basic variable that **first reaches 0** as entering variable increases (minimum ratio test)

Termination: Optimal BFS found (optimal z and decision variable values).

Worked example (from slides):

$$\begin{array}{ll} \max z = 8x_1 + 5x_2 & \rightarrow \text{Translated form with slacks } s_1, s_2: \\ \text{s.t. } x_1 + x_2 \leq 6 & s_1 = 6 - x_1 - x_2 \end{array}$$

$$9x_1 + 5x_2 \leq 45$$

$$x_1, x_2 \geq 0$$

$$s_2 = 45 - 9x_1 - 5x_2$$

$$z = 8x_1 + 5x_2$$

- **Starting BFS** (corner a): $x_1=0, x_2=0, s_1=6, s_2=45, z=0$
- **Iteration 1:** x_1 has largest positive coefficient (8). Min ratio: $\min(6/1, 45/9) = 5 \rightarrow s_2$ leaves. New corner d: $x_1=5, x_2=0$.
- **Iteration 2:** x_2 has positive coefficient. Min ratio: $\min(9/4, 1) \rightarrow s_1$ leaves. New corner c: $x_1=3.75, x_2=2.25$.
- **Optimal:** $z=41.25$ at $(3.75, 2.25)$ — no more positive coefficients.

Three special properties:

1. **Selection criterion** — choosing variable with largest coefficient doesn't guarantee fastest improvement (but it's computationally cheap)
2. **Unbounded LP** — if the entering variable can increase without bound (no basic variable reaches 0) $\rightarrow$ LP is unbounded ($z^* = +\infty$)
3. **Cycling** — rare; avoid by randomly choosing leaving variable when tie occurs

LP overview:

- Simplex: widely used; theoretically NP (worst case) but P in practice for most instances
- Karmarkar's interior point algorithm: provably polynomial (P), but less used in practice

2.4 ILP, MILP, BIP and Branch-and-Bound

Why ILP is harder than LP:

- LP optimum is at a corner point (BFS) — easy to find
- ILP optimum may be at an **interior point** of the LP feasible region — corner approach fails
- General ILP is **NP-complete**

LP relaxation:

- Replace all integer variables with continuous variables
- Solves faster; gives an **upper bound** $z_{LP^*} \geq z_{ILP^*}$ (for maximisation)
- If LP solution is all-integer $\rightarrow$ it IS the ILP optimum
- Rounding LP solution often gives infeasible or sub-optimal ILP solutions

Branch-and-Bound (B&B) Algorithm:

Principle: Implicitly enumerate all possible integer solutions by "divide and conquer."

General description (flowchart):

Initialise (LP relaxation of original problem)

↓

Bounding: solve LP relaxation of current subproblem $\rightarrow z_{UB}$

↓

- **SP5:** $z=40.56$ at $(4.44, 1)$ → branch on x_1 : SP6: $x_1 \geq 5$, SP7: $x_1 \leq 4$
- **SP7:** $z=37$ at $(4, 1)$ → *integer! Candidate.* $z_{LB}=37^*$
- **SP6:** $z=40$ at $(5, 0)$ → *integer! Better candidate.* $z_{LB}=40^*$
- **SP3:** $z=39$ at $(3, 3)$ → integer but $z=39 < 40$ → fathomed
- **Conclusion:** $z_{ILP^*} = 40$ at $(x_1, x_2) = (5, 0)$

2.5 Workflow: Solving Network Problems with (M)LP

```

Step 1: Analyse network problem → model as mathematical/graph problem
Step 2: Is it a known graph problem with an efficient graph algorithm?
Step 3: If NO → is it linear (or can be linearised)?
Step 4: If YES → assess complexity: LP or (M)ILP? #variables, #constraints,
problem structure
Step 5: If feasible → use commercial (M)ILP solver
        - Simple solver (Excel, Python) for 'easy' problems
        - Dedicated solver (Gurobi, CPLEX) for 'hard' problems
Step 6: Check and interpret results
  
```

Decision guide:

Problem type	Approach
Polynomial graph problem (e.g., shortest path, min-cost flow)	Use graph algorithm
LP (continuous, linear)	Simplex — handles 1000s of variables/constraints
ILP/MIP (integer, linear)	B&B or cutting plane — feasible up to ~hundreds of variables
Too complex for exact methods	Heuristics (Section 3)

3. Heuristics

3.1 Introduction

When to use heuristics:

- Problems too large for exact methods (LP, B&B)
- Some variables are discrete → combinatorial explosion
- Nonlinear objective or constraints
- **Trade-off:** computation time vs. solution quality

Goal: Find a **good** local optimum using **reasonable** means — not necessarily the global optimum.

Classification criteria:

Axis	Options
Scope	Generic ↔ Problem-specific
Method	Search (trajectory) ↔ Constructive (build up)
Randomness	Stochastic ↔ Deterministic
Quality guarantee	With ↔ Without

3.2 Penalties

Why penalties? Three problems with hard constraints in search heuristics:

1. Hard to find an initial feasible solution
2. Complex moves required to stay feasible
3. Feasible region may be disconnected (can't reach all parts)

Solution: Add a **penalty function $P(t)$** to the objective:

$$F_{\text{total}}(x) = F(x) + P(t) \quad \text{where } P(t) \text{ measures constraint violation}$$

Benefits:

- Converts hard constraints to soft constraints
- Simpler neighbourhood structure
- Can connect separated feasible regions
- Penalty weight can increase over time (low at start → easy exploration; high at end → forces feasibility)

3.3 Dedicated Heuristic Example: PON Design

Problem: Connect N hubs to a service node at minimum fiber digging cost.

Key observations:

- Not a standard MST (not all graph nodes needed — **Steiner Tree Problem** → NP-hard as ILP)
- Promote **path reuse**: assign zero cost to already-dug street segments

Final heuristic (problem-specific, deterministic, construction + search):

1. Calculate all shortest paths between leaf nodes (Dijkstra)
2. Connect two closest leaf nodes (Dijkstra path)
3. Assign zero cost to used street segments
4. Connect next closest leaf nodes
5. Repeat until all connected
6. **Post-optimisation:** remove small sub-trees and reconnect

Result on Ghent network (843 corners, 1281 segments, 44 hubs):

- Solution in 2.7 seconds; within 0.7% of other algorithms
- Post-optimisation gives 3% cost reduction

3.4 Steepest Descent

Algorithm:

```
(1) Initialise: select  $x_1 \in X$ ;  $f^* = f(x_1)$ ;  $x^* = x_1$ 
(2)  $n = 1$ ; STOP = FALSE
(3) While (not STOP):
(4)     Find best  $x'$  in neighbourhood  $N(x_n)$ 
(5)     If  $f(x') \leq f(x_n)$ :
(6)          $x_{n+1} = x'$ ;  $f^* = f(x')$ ;  $x^* = x'$ 
(7)     Else: STOP = TRUE
(8)      $n = n+1$ 
(9) Exit
```

Properties: Deterministic, generic, search heuristic

- Evaluates ALL neighbours at each step; picks the best
- **Fatal flaw:** Gets trapped in **local optima**
- **Remedy:** Restart from different initial solutions

3.5 Tabu Search (TS)

Principle: Improvement over Steepest Descent — can escape local optima by allowing non-improving moves.

Key ideas:

- At each step: choose the **best** neighbour (even if worse than current)
- **Tabu list:** Previously visited solutions (or their attributes) are "forbidden" to prevent cycling
 - **Explicit tabu list:** Stores full solutions permanently → memory intensive
 - **Attributive tabu list:** Stores changed attributes temporarily → more common; uses **tabu tenure** (time an attribute stays tabu)
- **Final solution:** Best solution found during the entire search (not necessarily the last)

Algorithm:

```
(1) Initialise:  $x_1 \in X$ ;  $f^* = f(x_1)$ ;  $x^* = x_1$ 
(2) While (stopping rule not fulfilled):
(3)     Find best  $x'$  in  $N(x_n)$  [excluding tabu solutions]
(4)      $x_{n+1} = x'$ 
(5)     If  $f(x') \leq f^*$ :  $f^* = f(x')$ ;  $x^* = x'$ 
(6)     Update tabu list
(7)     Update neighbourhood  $N(x_{n+1})$  based on tabu list
```

```
(8)      n = n+1
(9) Exit
```

Enhanced TS features:

Feature	Description
Stochastic TS	Examine only a random subset of $N(x)$ (for large neighbourhoods)
Reactive TS	Adjust tabu tenure dynamically to prevent cycling
Aspiration criteria	Override tabu status if solution is exceptionally attractive
Diversification	Force search into unexplored regions (e.g., new move types)
Intensification	Return to promising regions for deeper exploration

TS example (ring network design):

- Variables: binary (ring placed or not)
- Move: add or remove a ring
- Penalty: 5000 per uncovered node/link
- Tabu tenure: 3 (add ring), 6 (remove ring) — asymmetric because far more add options than remove
- Result: cost 35400 after 100 iterations

Classification: Deterministic, generic, search

3.6 Simulated Annealing (SA)

Analogy with metal annealing:

	Simulated Annealing	Metal Annealing
Goal	Minimise objective function	Minimise energy (perfect crystal)
Variables	Decision variables	Atom coordinates
Method	Random moves based on temperature T	Slow controlled cooling

Algorithm:

```
(1) Initialise:  $x_1 \in X$ ;  $f^* = f(x_1)$ ;  $x^* = x_1$ 
(2)  $n=1$ ; STOP=FALSE
(3) While (not STOP):
(4)   Draw  $x'$  at RANDOM from  $V(x_n)$  [random neighbour]
(5)   If  $f(x') \leq f(x_n)$ : accept ( $x_{n+1} = x'$ )
(6)     Update  $f^*$  and  $x^*$  if improved
(7)   Else: draw random  $p \in [0,1]$ 
(8)     If  $p \leq p_n$ : accept  $x'$  anyway ( $x_{n+1} = x'$ ) [Monte Carlo]
(9)     Else: reject ( $x_{n+1} = x_n$ )
```

```
(10)    n = n+1; evaluate STOP
(11) Exit
```

Acceptance probability (Boltzmann distribution):

$$p_n = \begin{cases} 1 & \text{if } f(x') \leq f(x_n) \\ e^{\{[f(x') - f(x_n)] / T\}} & \text{if } f(x') > f(x_n) \end{cases}$$

- High $T \rightarrow$ high $p_n \rightarrow$ many "uphill" moves accepted (exploration)
- Low $T \rightarrow$ low $p_n \rightarrow$ only improvements accepted (exploitation)
- Decreasing T over time $\rightarrow$ gradually more selective

Cooling schedule (geometric):

- After L trials (plateau length), multiply T by λ (cooling factor, $0 < \lambda < 1$): $T \rightarrow \lambda T$
- **Parameters to tune:** T_0 (initial temperature), L (plateau length), λ (cooling factor)
- **Stopping rule:** No improvement during last plateau (or temperature reaches 0)

SA vs. TS comparison:

- TS examines **all** N neighbours per step; SA examines **one random** neighbour per step
- For N neighbours: 1 TS step $\approx$ N SA steps in time, but SA explores more diverse paths
- For **large neighbourhoods**: SA often preferred (stochastic TS also good)

Classification: Stochastic, generic, search

3.7 Genetic Algorithms (GA)

Principle: Inspired by biological evolution — "survival of the fittest."

Key difference from TS/SA: Works with a **population** of solutions (not a single trajectory).

Solution encoding:

- Each solution is encoded as a **chromosome** (string of alleles)
- Simple: bit-string (for boolean variables)
- Custom encoding possible for better performance

Three genetic operators:

Operator	Description
Crossover	Combine two "parent" chromosomes at random cut points $\rightarrow$ two "child" chromosomes. Inherits good parts of both parents. Probability χ .
Mutation	Randomly flip one allele in a chromosome $\rightarrow$ "mutant." Drives search into new regions. Probability μ .

Operator	Description
Selection	Choose N chromosomes from the N+M population (N parents + M children+mutants) with probability proportional to fitness $f(x_i)$: $p_i = f(x_i) / \sum f(x_j)$. Survival of the fittest.

Algorithm:

```

(1) Initialise: population  $X^{(1)} = \{x_1^{(1)}, \dots, x_N^{(1)}\} \subset X$ 
       $f^* = \max\{f(x_i^{(1)})\}$ ,  $x^* = \operatorname{argmax}\{f(x_i^{(1)})\}$ 
(2) For  $g = 1, 2, \dots, G$ :
(3)   Crossover good parents (prob.  $\chi$ )  $\rightarrow$  children
(4)   Mutation of children and parents (prob.  $\mu$ )  $\rightarrow$  mutants
(5)   Update  $f^*$  and  $x^*$  if any new solution beats current best
(6)   Selection: choose N from N+M for next generation (fitness-
      proportional)
(7) Return  $f^*$ ,  $x^*$ 

```

GA vs. TS/SA:

- GA: **constructive** (crossover builds new solutions from parts) + **search** (mutation, selection)
- GA: works on building-block problems — good solution is a combination of good sub-solutions
- GA: naturally parallel (process population in parallel)
- Number of generations G specified in advance; hard to define stopping rule

Classification: Stochastic, generic, constructive + search

3.8 Concluding Remarks**Choosing a technique:**

- **Steepest Descent:** Simple; gets trapped in local optima; good baseline
- **Tabu Search:** Good for problems where full neighbourhood evaluation is feasible
- **Simulated Annealing:** Good for large neighbourhoods (random sampling)
- **Genetic Algorithms:** Good when good solutions are combinations of good sub-solutions

Technique-specific configuration:

- TS / SA: define (1) variables, (2) neighbourhood structure (the "move")
- GA: define (1) variables/encoding, (2) genetic operators (crossover, mutation, selection)

Key principle: Always add **problem-specific intelligence** to the neighbourhood structure or genetic operators. A generic heuristic with an intelligent, problem-aware move definition significantly outperforms a purely generic one.

PART 2: EXERCISES — SOLVED**Exercise Set A: LP Problems & Simplex Method**

A.1 LP Exercise 1

Problem:

$$\begin{aligned} \min z &= 2 + 5x_1 - 3x_2 \\ \text{s.t. } x_1 &\geq -5 \\ -x_1 + 2x_2 &\leq 15 \\ x_1 - x_2 &\geq -8 \\ x_1 \text{ urs, } x_2 &\geq 0 \end{aligned}$$

(a) Standard form transformation:

Step 1 (min $\rightarrow$ max): $\max z' = -2 - 5x_1 + 3x_2$

Step 2 ($x_1 \geq -5 \rightarrow$ multiply by -1): $-x_1 \leq 5$

Step 4 (x_1 urs: substitute $x_1 = y - 5, y \geq 0$):

This shifts x_1 so that $y = x_1 + 5 \geq 0$. Constraints become:

- $-x_1 \leq 5 \rightarrow y \geq 0$ $\checkmark$ (already a sign restriction)
- $-x_1 + 2x_2 \leq 15 \rightarrow -(y-5) + 2x_2 \leq 15 \rightarrow -y + 2x_2 \leq 10$
- $x_1 - x_2 \geq -8 \rightarrow (y-5) - x_2 \geq -8 \rightarrow y - x_2 \geq -3 \rightarrow$ multiply by -1 : $-y + x_2 \leq 3$ (i.e., $x_2 \leq y+3$)
- Objective: $z = -23 + 5y - 3x_2 \rightarrow \max z' = 23 - 5y + 3x_2$

Standard form:

$$\begin{aligned} \max z' &= 23 - 5y + 3x_2 \\ \text{s.t. } -y + 2x_2 &\leq 10 & [C1] \\ -y + x_2 &\leq 3 & [C2] \\ y, x_2 &\geq 0 \end{aligned}$$

(b) Simplex solution:

Add slack variables s_1, s_2 :

$$\begin{aligned} -y + 2x_2 + s_1 &= 10 \\ -y + x_2 + s_2 &= 3 \end{aligned}$$

Starting BFS: $y=0, x_2=0, s_1=10, s_2=3, z'=23$.

Iteration 1: Objective: $z' = 23 - 5y + 3x_2$. Coefficient of x_2 is $+3$ (positive) $\rightarrow x_2$ enters.

- Min ratio: C1: $10/2=5$; C2: $3/1=3 \rightarrow s_2$ leaves (min ratio = 3)

From C2: $x_2 = 3 + y - s_2$

Substitute into C1: $-y + 2(3+y-s_2) + s_1 = 10 \rightarrow y + s_1 - 2s_2 = 4$

Update z': $z' = 23 - 5y + 3(3+y-s_2) = 32 - 2y - 3s_2$

Optimality check: Nonbasic variables y and s_2 have coefficients -2 and -3 (both ≤ 0) $\rightarrow$ **OPTIMAL!**

Optimal BFS: $y=0, s_2=0 \rightarrow x_2=3, s_1=4$.

$z' = 32 \rightarrow z_{\min} = -z' + 23 = \dots$ actually: $z = -23 + 5y - 3x_2 = -23 + 0 - 9 = -32$

And $y=0 \rightarrow x_1 = y-5 = -5$

Answer: $z^*_{LP} = -32$ at $(x^*_1, x^*_2) = (-5, 3)$

(c) Graphical verification:

The feasible region (in x_1, x_2 space with x_1 urs, $x_2 \geq 0$):

- $x_1 \geq -5$ (vertical line $x_1=-5$)
- $x_2 \leq (15+x_1)/2$ (upper boundary from C2)
- $x_2 \leq x_1+8$ (upper boundary from C3: $x_1-x_2 \geq -8$)
- $x_2 \geq 0$

Corner points: $A=(-5, 3)$, $B=(-1, 7)$, $C=(-5, 5)$

Corner	$z = 2+5x_1-3x_2$
$A=(-5, 3)$	$2-25-9 = -32$
$B=(-1, 7)$	$2-5-21 = -24$
$C=(-5, 5)$	Only on boundary of C1 & $x_1=-5$; C3 gives $x_2 \leq 3$ when $x_1=-5 \rightarrow C$ is infeasible

The binding constraint $x_2 \leq x_1+8 = 3$ at $x_1=-5$ means x_2 cannot exceed 3 at $x_1=-5$. So $C=(-5,5)$ is **infeasible** (violates $x_2 \leq x_1+8$).

Minimum $z = -32$ at $(-5, 3)$ ✓

A.2 LP Exercise 2

Problem (already in standard form):

$$\begin{array}{ll}
 \max & z = x_1 + x_2 \\
 \text{s.t.} & x_1 + x_2 \leq 4 \quad [C1] \\
 & x_1 \leq 3 \quad [C2] \\
 & x_2 \leq 3 \quad [C3] \\
 & x_1, x_2 \geq 0
 \end{array}$$

Add slacks s_1, s_2, s_3 :

Starting BFS: $x_1=x_2=0, s_1=4, s_2=3, s_3=3, z=0$.

Iteration 1: Both x_1 and x_2 have coefficient $+1$. Choose x_1 .

- Ratios: C1: $4/1=4$; C2: $3/1=3$; C3: no $x_1 \rightarrow s_3$ irrelevant
- Min ratio = 3 (C2) $\rightarrow s_2$ leaves

From C2: $x_1 = 3 - s_2$.

Update z: $z = (3-s_2) + x_2 = 3 + x_2 - s_2$

Update C1: $(3-s_2)+x_2+s_1=4 \rightarrow x_2 + s_1 - s_2 = 1$

Iteration 2: $z = 3 + x_2 - s_2$. Coefficient of x_2 is +1 $\rightarrow x_2$ enters.

- Ratios: C1: $1/1=1$ (from $x_2+s_1=1$); C3: $3/1=3$
- Min ratio = 1 (C1) $\rightarrow s_1$ leaves

From C1: $x_2 = 1 + s_2 - s_1$.

Update z: $z = 3 + (1+s_2-s_1) - s_2 = 4 - s_1$

Current solution: $x_1=3, x_2=1, s_3=3-x_2=2$.

Optimality check: $z = 4 - s_1$. Nonbasic: s_1 (coeff -1), s_2 (coeff 0).

- s_1 has negative coefficient $\rightarrow$ can't improve
- s_2 has coefficient 0 $\rightarrow$ can enter basis without changing z $\rightarrow$ **infinite optimal solutions!**

Entering s_2 (with $s_1=0$): traces the segment from (3,1) to (1,3) along $x_1+x_2=4$.

Answer: $z^*_{LP} = 4$ at all points $(x^*_1, x^*_2) = \lambda \cdot (1,3) + (1-\lambda) \cdot (3,1), \lambda \in [0,1]$

(Infinitely many optimal solutions along the edge $x_1+x_2=4$ between the two corner points)

A.3 LP Exercise 3

Problem:

```

min z = x1 - 2x2
s.t.  10 + x1 + x3 ≥ x2      →   x2 - x1 - x3 ≤ 10
      -4 ≤ x1 + x2 ≤ 6      →   x1 + x2 ≤ 6 (lower bound redundant with
x1, x2 urs)
      x1 urs, x2 urs, x3 ≥ 0

```

Why $z^* = -\infty$ (unbounded):

With x_1 and x_2 unrestricted in sign, consider the sequence:

- $x_1 = -t, x_2 = 6+t$ (so $x_1+x_2 = 6 \leq 6 \checkmark$), $x_3 = 2t-4 \geq 0$ for $t \geq 2$

Check constraint 1: $x_2 - x_1 - x_3 = (6+t) - (-t) - (2t-4) = 6+t+t-2t+4 = 10 \leq 10 \checkmark$

Objective: $z = x_1 - 2x_2 = -t - 2(6+t) = -3t - 12 \rightarrow -\infty$ as $t \rightarrow \infty$

The Simplex detects this: when trying to increase a variable with positive objective coefficient, no basic variable ever reaches zero (unbounded feasible direction).

Answer: $z^*_{LP} = -\infty$ (unbounded problem)

The objective can decrease without bound by increasing x_2 while correspondingly decreasing x_1 and increasing x_3 .

Exercise Set B: MILP Problems & Branch-and-Bound

B.1 B&B Exercise 1

Problem (ILP):

$$\begin{aligned} \max \quad & z = x_1 + x_2 \\ \text{s.t.} \quad & 5x_1 + 3x_2 \leq 30 \\ & -x_1 + x_2 \leq 8 \\ & x_1, x_2 \geq 0, \text{ integer} \end{aligned}$$

LP relaxation (SP1):

Corner points of LP feasible region:

- Origin: $z=0$
- $5x_1+3x_2=30$ and $x_1=0$: $x_2=10 \rightarrow z=10$
- $-x_1+x_2=8$ and $x_1=0$: $x_2=8 \rightarrow z=8$
- Intersection $5x_1+3x_2=30$ and $-x_1+x_2=8$: From second: $x_2=x_1+8$. Sub: $5x_1+3(x_1+8)=30 \rightarrow 8x_1=6 \rightarrow x_1=3/4$, $x_2=35/4=8.75$

Solve graphically: z_{LP}^* at corner of $5x_1+3x_2=30$ and $-x_1+x_2=8$ is NOT optimal for $\max z=x_1+x_2$ (isoprofit lines at 45°).

Optimal LP corner: check all. The highest $z = x_1+x_2 = x_1+(30-5x_1)/3 = 10-(2/3)x_1 \rightarrow$ decreases with $x_1 \rightarrow$ best at $x_1=0$: $z=10$ at $(0,10)$.

But also check $(3/4, 35/4)$: $z = 3/4+35/4 = 38/4 = 9.5$.

LP optimal: $(0, 10)$, $z_{LP}^* = 10^*$ $\rightarrow$ upper bound $z_{ILP}^* \leq 10$. $x_2=10$ is integer! $x_1=0$ is integer! $\rightarrow z_{ILP} = 10?$

Wait:* check feasibility: $5(0)+3(10)=30 \leq 30 \checkmark$, $-0+10=10 \leq 8$? **NO!** $10 > 8 \rightarrow (0,10)$ is INFEASIBLE.

Recheck LP relaxation more carefully. The active constraints at optimum:

Isoprofit lines $z=x_1+x_2$ (slope -1). Move isoprofit upward.

Constraints: region bounded by $5x_1+3x_2 \leq 30$, $-x_1+x_2 \leq 8$, $x_1, x_2 \geq 0$.

Corner at intersection of $-x_1+x_2=8$ and $x_1=0$: $(0,8)$ — feasible? $5(0)+3(8)=24 \leq 30 \checkmark$. $z=8$.

Corner at $(0,8)$ and $5x_1+3x_2=30$ and $-x_1+x_2=8$ intersection $(3/4, 35/4)$: $z=9.5$.

Corner at $5x_1+3x_2=30$ and $x_2=0$: $(6,0)$. $z=6$.

LP optimal: $z_{LP}^* = 9.5$ at $(0.75, 8.75)$. Upper bound: $z_{ILP}^* \leq 9$ (since coefficients are integer, z is integer too).

Branch on $x_1 (= 0.75)$:

- SP2: add $x_1 \geq 1$

- SP3: add $x_1 \leq 0$

SP3 ($x_1=0$): $5(0)+3x_2 \leq 30 \rightarrow x_2 \leq 10$; $-0+x_2 \leq 8 \rightarrow x_2 \leq 8$. LP: $x_2=8, z=8$. $x_2=8$ integer! *Candidate: $z_{LB}=8$.**

SP2 ($x_1 \geq 1$): New intersection: $5x_1+3x_2=30, x_2=x_1+8 \rightarrow 5x_1+3(x_1+8)=30 \rightarrow 8x_1=6 \rightarrow x_1=0.75$... still <1 .

With $x_1 \geq 1$: corner at $x_1=1$: $5+3x_2 \leq 30 \rightarrow x_2 \leq 25/3 \approx 8.33$; $-1+x_2 \leq 8 \rightarrow x_2 \leq 9$. LP: $(1, 25/3), z \approx 9.33$. Branch on x_2 .

SP2a: $x_2 \geq 9$ with $x_1 \geq 1$: $5x_1+3(9) \leq 30 \rightarrow 5x_1 \leq 3 \rightarrow x_1 \leq 0.6$. But $x_1 \geq 1$: **INFEASIBLE**.

SP2b: $x_2 \leq 8$ with $x_1 \geq 1$: $5x_1+3x_2 \leq 30, -x_1+x_2 \leq 8, x_1 \geq 1, x_2 \leq 8$. LP optimal: try $(1,8)$: $5+24=29 \leq 30 \checkmark, -1+8=7 \leq 8 \checkmark, z=9$. **Integer!** New candidate: $z_{LB}=9$. Fathomed (integer).

No more subproblems.

Answer: $z^*_{ILP} = 9$ at $(x^*_1, x^*_2) = (1, 8)$

B.2 B&B Exercise 2

Problem (MILP):

$$\begin{aligned} \max \quad & z = 2x_1 + x_2 \\ \text{s.t.} \quad & 5x_1 + 2x_2 \leq 8 \\ & x_1 + x_2 \leq 3 \\ & x_1, x_2 \geq 0; \quad x_1 \text{ integer, } x_2 \text{ real} \end{aligned}$$

LP relaxation (SP1):

Intersection of $5x_1+2x_2=8$ and $x_1+x_2=3$: $x_2=3-x_1 \rightarrow 5x_1+2(3-x_1)=8 \rightarrow 3x_1=2 \rightarrow x_1=2/3, x_2=7/3$.

$z=2(2/3)+7/3=11/3 \approx 3.67$. Upper bound $z_{MILP}^* \leq 3.67$.

$x_1=2/3$ is not integer. Branch on x_1 .

SP2: $x_1 \leq 0$ (i.e., $x_1=0$, since $x_1 \geq 0$): $2x_2 \leq 8 \rightarrow x_2 \leq 4$; $x_2 \leq 3$. LP: $x_2=3, z=3$. $x_1=0$ (integer). *Candidate: $z_{LB}=3$.**

SP3: $x_1 \geq 1$: $5+2x_2 \leq 8 \rightarrow x_2 \leq 1.5$; $1+x_2 \leq 3 \rightarrow x_2 \leq 2$. LP: $x_1=1, x_2=1.5, z=2+1.5=3.5$.

$x_1=1$ is integer! $x_2=1.5$ is real (allowed). *Better candidate: $z_{LB}=3.5$.** Fathomed (x_1 is integer, x_2 is real $\rightarrow$ solution is feasible for MILP).

SP2 candidate ($z=3$) < SP3 ($z=3.5$) — no more subproblems.

Answer: $z^*_{MILP} = 3.5$ at $(x^*_1, x^*_2) = (1, 1.5)$

B.3 B&B Exercise 3

Problem (ILP — same as B.2 but BOTH variables integer):

$$\begin{aligned} \max \quad & z = 2x_1 + x_2 \\ \text{s.t.} \quad & 5x_1 + 2x_2 \leq 8 \end{aligned}$$

$$\begin{aligned} x_1 + x_2 &\leq 3 \\ x_1, x_2 &\geq 0, \text{ both integer} \end{aligned}$$

SP1 LP relaxation: $z_{LP^*} = 11/3 \approx 3.67$ at $(2/3, 7/3)$. Branch on x_1 .

SP2: $x_1 \leq 0$ ($x_1 = 0$): $x_2 \leq 4$ and $x_2 \leq 3 \rightarrow$ LP: $x_2 = 3$. $z = 3$. Integer! *Candidate* $(0, 3)$, $z_{LB} = 3^*$.

SP3: $x_1 \geq 1$: LP: $x_1 = 1$, $x_2 = 1.5$, $z = 3.5$. $x_2 = 1.5$ not integer. Branch on x_2 .

SP3a: $x_1 \geq 1$, $x_2 \geq 2$: $5 + 4 = 9 > 8 \rightarrow$ **INFEASIBLE**.

SP3b: $x_1 \geq 1$, $x_2 \leq 1$: $5 + 2 \leq 8 \checkmark$, $1 + 1 \leq 3 \checkmark$. LP: $x_1 = 1$, $x_2 = 1$, $z = 3$. Integer! *New candidate* $(1, 1)$. $z_{LB} = 3^*$ (tie with existing).

No more subproblems.

Both $(0, 3)$ and $(1, 1)$ are optimal with $z = 3$.

Answer: $z^*_{ILP} = 3$ at $(x^*_1, x^*_2) = (0, 3)$ OR $(1, 1)$ — two optimal solutions

Exercise Set C: Network Optimisation Problems

C.1 Exercise 1: Bandwidth Allocation to Services

Problem: N services $\{S_i\}$ on a transatlantic link with capacity C_{tot} . Time horizon divided into M equal slots. Service i : begins at slot m_i^b , ends at slot m_i^e , requires bandwidth B_i , yields profit P_i if accepted. Maximise total profit.

(a) MILP formulation:

Decision variables: $x_i \in \{0, 1\}$ for $i = 1, \dots, N$
 ($x_i = 1$ if service S_i is accepted, 0 otherwise)

Objective:

$$\max \quad z = \sum_i P_i x_i$$

Constraints (capacity respected in every time slot m):

$$\forall m \in \{1, \dots, M\}: \quad \sum_{\{i: m_i^b \leq m \leq m_i^e\}} B_i x_i \leq C_{tot}$$

Variable type: $x_i \in \{0, 1\}$ (BIP) $\rightarrow$ This is a **Binary Integer Programming** problem.

This is in fact a **Weighted Interval Scheduling / Knapsack-type BIP**.

(b) Reducing number of constraints when $M \gg N$:

Key insight: The capacity constraint can only change when a service **starts or ends**. Between two consecutive events (start/end of any service), the set of active services does not change — so we only need one constraint per "interval between events."

Reduction: Instead of M constraints (one per slot), use one constraint per distinct **critical time interval** between events. There are at most **$2N$ event times** (N starts + N ends) $\rightarrow$ at most **$2N - 1$ intervals** $\rightarrow$ at most **$2N$ constraints** instead of M .

Since $M \gg N$, this reduces the number of constraints from M to $O(N)$ — a drastic reduction.

(c) Solution method:

The problem is a **BIP (Binary Integer Program)**. Approaches:

1. **Branch-and-Bound** with LP relaxation (relax x_i to $[0,1]$)
2. **Greedy heuristic** (sort services by profit-per-bandwidth ratio, greedily accept)
3. **Dynamic programming** (if service intervals are nested or structured)
4. Commercial solver (Gurobi/CPLEX) for exact solution

C.2 Exercise 2: Routing with Limitations

The Physical Scenario

You have a **telecom network** — think of it as a set of routers or switching nodes connected by links (fibre cables, for example). Multiple customers or services want to send traffic across this network simultaneously. Your job is to decide **which link carries how much of whose traffic**, so that:

1. You minimise the total cost of carrying all traffic
2. No link is overloaded (capacity respected)
3. Every flow arrives within an acceptable delay (real-time service)

What is a "Commodity"?

A **commodity** is one specific traffic demand: a pair (source, destination) with a given amount of traffic in Gbit/s. For example: "3 Gbit/s must get from v_1 to v_2 " is one commodity. "8 Gbit/s must get from v_2 to v_6 " is a different commodity.

The key word is "multi-commodity": many such demands coexist **simultaneously**, and they **share the same physical links**. This sharing is what makes the problem hard — if you only had one demand, it would just be a shortest-path problem.

Every Symbol Explained

The network structure:

Symbol	Type	Plain meaning
V	set	All nodes (routers, switches) in the network
A	set	All directed arcs (links with a direction) — each physical cable counts as two arcs: one in each direction
$G = (V, A)$	graph	The full network

Parameters (given, not decided):

Symbol	Type	Plain meaning
$c(a)$	number (€/Gbit/s)	Cost per unit of traffic sent along arc a . Sending 1 Gbit/s over arc a costs $c(a)$ euros.
$u(a)$	number (Gbit/s)	Capacity of arc a — the maximum total traffic (all commodities combined) that this link can carry
$d(a)$	number (ms)	Propagation delay of arc a — how many milliseconds it takes for a packet to traverse this link
$d_{\{ij\}}$	number (Gbit/s)	Demand : how many Gbit/s must be sent from node i to node j . If $d_{\{ij\}}=0$, no traffic is required on that pair.
$D_{\max}$	number (ms)	Maximum acceptable end-to-end delay for any flow (the real-time service requirement)

Index sets (derived from the problem):

Symbol	Plain meaning
K	The set of all commodities — all (source, destination) pairs (i,j) where $d_{\{ij\}} > 0$. Each commodity k has a source node s_k , a destination node t_k , and a demand $d_k = d_{\{s_k,t_k\}}$.
$I(v)$	All arcs arriving at node v (the links that point INTO v)
$\Gamma(v)$	All arcs leaving node v (the links that point OUT OF v)

Decision variables (what the solver decides):

Symbol	Type	Plain meaning
$f^k(a)$	real ≥ 0	The amount of traffic of commodity k (in Gbit/s) routed through arc a . This is the core unknown.

Note: There is one variable $f^k(a)$ for every combination of commodity k AND arc a . For a network with 7 nodes, ~15 arcs, and 10 commodities, that's ~150 variables. All are continuous (real-valued) → this is an **LP**.

The Objective Function — In Plain English

$$\min \sum_{a \in A} c(a) \cdot \sum_{k \in K} f^k(a)$$

Step by step what this says:

- $\sum_{k \in K} f^k(a)$ — sum all commodity flows on arc a :
 → This gives the **total traffic on arc a** (Gbit/s), regardless of which commodity it belongs to.
 → E.g. if arc a carries 3 Gbit/s of commodity " $v_1 \rightarrow v_2$ " and 5 Gbit/s of commodity " $v_3 \rightarrow v_4$ ", the total is 8 Gbit/s.
- $c(a) \cdot \sum_{k \in K} f^k(a)$ — multiply total traffic on arc a by its cost per Gbit/s:
 → This gives the **cost of using arc a** given how much you route through it.
 → Cost is proportional to traffic (assumption of the LP model).
- $\sum_{a \in A} [\dots]$ — sum over every arc in the network:
 → This gives the **total network-wide cost**.

Bottom line: The objective = "minimise the total money paid to run all the traffic across all links." You want to use cheap arcs and avoid expensive ones, while still routing all demanded traffic.

The Constraints — In Plain English

Constraint 1: Flow Conservation

$$\forall v \in V, \forall k \in K: \sum_{a \in I(v)} f^k(a) - \sum_{a \in \Gamma(v)} f^k(a) = b_k(v)$$

Where:

$$b_k(v) = \begin{cases} -d_k & \text{if } v = s_k \quad (\text{source node of commodity } k) \\ +d_k & \text{if } v = t_k \quad (\text{destination node of commodity } k) \\ 0 & \text{everywhere else} \end{cases}$$

What this says: At every node v , for every commodity k , **traffic must be conserved** — you can't create or lose traffic at a transit node.

At a transit node ($b_k(v) = 0$): whatever volume of commodity k 's traffic enters, the same volume must leave. Nothing is stored or generated. $\sum_{\text{arriving}} = \sum_{\text{leaving}}$.

At the source ($b_k(v) = -d_k$, negative): the source INJECTS d_k Gbit/s into the network. So outflow exceeds inflow by d_k . $\sum_{\text{leaving}} - \sum_{\text{arriving}} = d_k$.

At the destination ($b_k(v) = +d_k$, positive): the destination ABSORBS d_k Gbit/s. Inflow exceeds outflow by d_k . $\sum_{\text{arriving}} - \sum_{\text{leaving}} = d_k$.

Why one constraint per (node, commodity) pair? Because flows of different commodities must be tracked separately — 3 Gbit/s of " $v_1 \rightarrow v_2$ " traffic and 5 Gbit/s of " $v_2 \rightarrow v_6$ " traffic passing through the same router must

each be tracked independently (they go to different destinations).

Constraint 2: Capacity

$$\forall a \in A: \quad \sum_{k \in K} f^k(a) \leq u(a)$$

What this says: On each link a , the **combined traffic from ALL commodities** must not exceed the link's physical capacity $u(a)$.

This is the coupling constraint — it's the reason you cannot solve each commodity independently. If commodity " $v_1 \rightarrow v_2$ " and commodity " $v_3 \rightarrow v_4$ " both want to use the same link, they compete for capacity. The routing decision for one commodity affects what's left for others.

Constraint 3: Delay (max path delay $\leq D_{\max}$)

The problem requires that **every packet** of commodity k arrives within $D_{\max}$ — this is a *maximum* path delay constraint, not an average.

Why a simple linear formula like $\sum d(a) \cdot f^k(a) \leq D_{\max} \cdot d_k$ is NOT correct:

That would only constrain the *weighted average* delay across all paths used. Example with $D_{\max} = 30$ ms:

Path used	Delay	Flow
$v_1 \rightarrow v_7 \rightarrow v_3$	10 ms	1 Gbit/s
$v_1 \rightarrow v_2 \rightarrow v_3$	50 ms	1 Gbit/s

Average = $(1 \times 10 + 1 \times 50) / 2 = 30$ ms ≤ 30 ✓ — but path 2 has delay 50 ms $> D_{\max}$. Real-time packets on that path arrive late. The average constraint is too weak.

The correct formulation — pre-processing the graph:

Before writing the LP, remove any arc $a = (u \rightarrow v)$ that *cannot* be part of any delay-feasible path for commodity k . Specifically, keep arc a for commodity k only if:

$$\text{min_delay}(s_k \rightarrow u) + d(a) + \text{min_delay}(v \rightarrow t_k) \leq D_{\max}$$

(compute both shortest-delay distances with Dijkstra on $d(a)$ weights). Then set:

$$\forall a \text{ not satisfying the above: } f^k(a) = 0$$

After this preprocessing, any flow that satisfies flow conservation in the reduced graph can be decomposed into paths — and every such path has total delay $\leq D_{\max}$ by construction. The LP then implicitly enforces the maximum delay constraint through the arc restriction, without needing an explicit linear delay inequality.

Constraint 4: Non-negativity

$$\forall a \in A, \forall k \in K: f^k(a) \geq 0$$

Traffic flows can only be non-negative — you can't send "negative" traffic.

Full Formulation at a Glance

GIVEN: Network $G=(V,A)$, costs $c(a)$, capacities $u(a)$, delays $d(a)$
 Demands d_{ij} for all (i,j) , delay limit $D_{\max}$

DECIDE: $f^k(a) \geq 0$ for all arcs $a \in A$ and all commodities $k \in K$

MINIMISE:

$$\begin{aligned} \text{Total cost} &= \sum_{a \in A} c(a) \cdot [\text{total traffic on arc } a] \\ &= \sum_{a \in A} c(a) \cdot \sum_{k \in K} f^k(a) \end{aligned}$$

SUBJECT TO:

- [C1] Flow conservation at every node, for every commodity
- [C2] Total traffic on every arc $\leq$ arc capacity
- [C3] $f^k(a) = 0$ for arcs not on any delay-feasible path for k (max path delay $\leq D_{\max}$)
- [C4] All flows non-negative

This is a **Linear Program (LP)**: objective and all constraints are linear in $f^k(a)$, and all variables are continuous (real-valued Gbit/s amounts).

(a) Single or Multi-Commodity?

Multi-commodity. In the demand matrix there are 9 non-zero entries — that means 9 different (source, destination) pairs that need traffic simultaneously. Each is a separate commodity because:

- Traffic for $v_1 \rightarrow v_2$ and traffic for $v_3 \rightarrow v_4$ are different "goods" — they come from different places and go to different places
- You must track them separately — otherwise you couldn't enforce that "3 Gbit/s actually arrives at v_2 from v_1 " vs "5 Gbit/s arrives at v_4 from v_1 "
- They share capacity on common links $\rightarrow$ they interact

A **single-commodity** problem would mean every unit of traffic is identical (same origin and destination for all traffic). That reduces to a simple min-cost flow problem solvable with a graph algorithm.

(c) Four Special Cases — Why Each One Simplifies

Case DelCapUnl — No delay limit, no capacity limit ($D_{\max}=\infty$, $u(a)=\infty$):

If there is no capacity constraint [C2], then the routing decisions for different commodities do not affect each other at all. No link can be overloaded, so commodity k 's routing cannot "block" commodity j 's routing.

With no delay constraint [C3] either, each commodity k just needs to minimise its own cost: route all d_k Gbit/s along the cheapest possible path from s_k to t_k .

→ **This is exactly what Dijkstra's shortest-path algorithm does** (with arc costs $c(a)$ as weights). Run Dijkstra once per commodity.

→ **YES, standard graph problem ✓**

Case DelUnl — No delay limit, capacity constraints remain ($D_{\max}=\infty$):

Remove constraint [C3] from the formulation. Now you have the classic **Multi-Commodity Min-Cost Flow (MCMF)** problem.

The coupling through capacity [C2] still exists: commodities share links, so you can't optimise them independently. You can't use Dijkstra. You need the full LP.

For single-commodity this would be solvable with a standard min-cost flow graph algorithm, but with multiple commodities sharing capacity the problem must be solved as a system.

→ **NO, use LP (Simplex) ✗**

Case CapUnl — Capacity unlimited, delay constraints remain ($u(a)=\infty$):

Remove constraint [C2]. Now no link can overflow, so again the commodities don't interact — each can be solved independently.

But constraint [C3] remains: each commodity must arrive within $D_{\max}$. For each commodity k independently, the problem is: "find the minimum-cost routing from s_k to t_k such that the average delay $\leq D_{\max}$."

This is called the **Constrained Shortest Path (CSP)** problem — find the cheapest path that also meets a delay budget. This is **NP-hard** in general (no known polynomial algorithm). You can't just run Dijkstra, because minimising cost and minimising delay are two conflicting objectives and you need a specific trade-off between them.

→ **NO, requires special algorithms or LP relaxation ✗**

Case Orig — Both constraints active:

This is the full problem. Capacity couples the commodities, and delay constrains each one. It remains an LP (all variables continuous), solvable with Simplex — but not decomposable into independent graph problems.

→ **NO, use LP (Simplex) ✗**

Summary table:

Case	Constraints active	Commodities independent?	Standard graph?	Method
DelCapUnl	neither	YES	YES	Dijkstra per commodity
DelUnl	only capacity	NO (coupled by capacity)	NO	LP — Simplex
CapUnl	only delay	YES	NO (CSP is NP-hard)	LP per commodity / heuristic
Orig	both	NO	NO	LP — Simplex

The key pattern: Capacity constraint = coupling between commodities = cannot solve independently. No capacity = each commodity is its own sub-problem. Delay on top = each sub-problem is harder than a plain shortest path.

C.3 Exercise 3: Clustering in MANET (DS and CDS)

Definitions:

- **Dominating Set (DS):** Subset $S \subseteq V$ such that every node is either in S or has a neighbour in S
- **Connected Dominating Set (CDS):** DS where the induced subgraph $G[S]$ is connected

(a) MILP formulation for minimal DS:

Decision variables: $x_v \in \{0,1\}$ for each $v \in V$

($x_v = 1$ if node $v \in S$, 0 otherwise)

Objective:

$$\min \sum_{v \in V} x_v$$

Constraints (every node must be dominated):

$$\forall v \in V: x_v + \sum_{u \in N(v)} x_u \geq 1$$

(node $v \in S$, OR at least one neighbour of v is in S)

Variable type: $x_v \in \{0,1\} \rightarrow$ **BIP problem**

(b) MILP formulation for minimal CDS:

Same DS variables $x_v \in \{0,1\}$, plus **flow variables** to enforce connectivity.

Let $r \in V$ be an arbitrary root node (fixed in S : $x_r = 1$).

For connectivity: route 1 unit of flow from r to every other node v with $x_v=1$.

Additional variables: $f_{uv} \geq 0$ for each directed arc (u,v) (flow on edge $u \rightarrow v$)

Additional constraints:

Flow conservation (each non-root S-node receives 1 unit):

$$\forall v \in V \setminus \{r\}: \quad \sum_{\{u: (u,v) \in E\}} f_{uv} - \sum_{\{u: (v,u) \in E\}} f_{vu} = x_v$$

Root sends flow to all S-members:

$$\sum_{\{u: (r,u) \in E\}} f_{ru} - \sum_{\{u: (u,r) \in E\}} f_{ur} = \sum_{\{v \neq r\}} x_v$$

Flow only on edges within S (big-M formulation):

$$\begin{aligned} \forall (u,v) \in E: \quad f_{uv} &\leq |V| \cdot x_u \\ f_{uv} &\leq |V| \cdot x_v \end{aligned}$$

(c) Minimal DS → standard graph problem?

NO. Minimum Dominating Set is **NP-hard** (NP-complete decision version). No efficient polynomial algorithm is known for general graphs.

Greedy heuristic for minimal DS:

1. Initialise: $S = \emptyset$, uncovered = V
2. While uncovered $\neq \emptyset$: a. For each candidate node $v \notin S$, compute coverage: $|\{v\} \cup N(v) \cap \text{uncovered}|$ b. Select v^* with **maximum coverage** c. Add v^* to S d. Remove v^* and all $N(v^*)$ from uncovered
3. Return S

Scalability: $O(|V|^2)$ — scales well for large MANETs

Optimality: Not guaranteed; greedy approximation within $O(\log \Delta)$ of optimal (where $\Delta = \max \text{degree}$)

(d) Minimal CDS → standard graph problem?

NO. Minimum Connected Dominating Set is also **NP-hard**. Even harder than DS due to connectivity requirement.

(e) Greedy heuristic for minimal DS (detailed):

```
Algorithm: Greedy_DS( $G=(V,E)$ )
Input: Undirected graph  $G$ 
Output: Dominating set  $S$ 
```

```
 $S = \emptyset$ 
dominated =  $\emptyset$ 
```

```

While dominated  $\neq V$ :
  For each  $v \in V$ :
    gain( $v$ ) =  $|\{v\} \cup N(v) \setminus \text{dominated}|$  // nodes  $v$  would newly
    dominate
   $v^* = \operatorname{argmax}_{v \notin S} \text{gain}(v)$ 
   $S = S \cup \{v^*\}$ 
  dominated = dominated  $\cup \{v^*\} \cup N(v^*)$ 

Return  $S$ 

```

Discussion:

- **Scalability:** $O(|V|^2)$ — each iteration $O(|V|)$ with $|V|$ iterations. Scales to large MANETs.
- **Optimality:** Sub-optimal in general. For the example in the slides (6 nodes), the greedy may find a non-minimal DS. Example in (c) of the figure: optimal CDS has 2 nodes (the two connected red nodes), but greedy DS might find 3 nodes.

(f) Greedy heuristic for minimal CDS:

```

Algorithm: Greedy_CDS( $G=(V,E)$ )
1. Run Greedy_DS above → initial DS  $S_{DS}$  (may not be connected)
2. Connect  $S_{DS}$ :
  While the induced subgraph  $G[S_{DS}]$  is not connected:
    Find connected components  $C_1, C_2, \dots$  of  $G[S_{DS}]$ 
    For each pair  $(C_i, C_j)$ :
      Find the shortest path  $P$  between  $C_i$  and  $C_j$  in  $G$ 
      Add all nodes of the shortest inter-component path to  $S_{DS}$ 
3. Return  $S_{DS}$  (now a CDS)

```

Alternative: use a **spanning-tree-based** approach:

1. Find a spanning tree T of G
2. All **internal nodes** of T (degree ≥ 2 in T) form a connected dominating set
3. Remove unnecessary nodes (iterative reduction)

Discussion:

- **Scalability:** $O(|V|^2 \log |V|)$ — scalable
- **Optimality:** Not optimal in general; typically yields a CDS within a small constant factor of optimal in practice

PART 3: QUICK REFERENCE CARDS

Simplex Cheat Sheet

Standard form: $\max z = c^T x, \quad Ax \leq b, \quad x \geq 0$

1. Add slack variables $\rightarrow Ax + Is = b$
2. Initial BFS: $x=0, s=b$ (all slacks basic)
3. Entering variable: highest positive coefficient in z-row
4. Leaving variable: minimum ratio b_i/a_{ij} (only for $a_{ij} > 0$)
5. Pivot: row operations to express z and basics in terms of non-basics
6. Repeat until no positive z-coefficients $\rightarrow$ OPTIMAL
Or: entering variable has no positive $a_{ij} \rightarrow$ UNBOUNDED

Branch-and-Bound Cheat Sheet

For max ILP:

1. Solve LP relaxation $\rightarrow z_{LP^*}, x_{LP^*}$
2. If x_{LP^*} is all-integer: DONE ($z_{ILP^*} = z_{LP^*}$)
3. If not: branch on fractional variable $x_i = f$
 $\rightarrow$ SP_left: add $x_i \leq \lfloor f \rfloor$
 $\rightarrow$ SP_right: add $x_i \geq \lceil f \rceil$
4. For each subproblem:
FATHOM if: (a) infeasible
(b) $z_{UB} \leq z_{LB^*}$ (can't beat incumbent)
(c) $z_{UB} > z_{LB^*}$ AND solution integer $\rightarrow$ new incumbent
5. STOP when no active subproblems remain

Heuristic Selection Guide

Problem type	$\rightarrow$ Recommended approach
LP (continuous, linear)	$\rightarrow$ Simplex (exact)
ILP/MIP, small-medium	$\rightarrow$ Branch-and-Bound (exact)
ILP/MIP, too large for B&B	$\rightarrow$ Heuristics
Nonlinear / non-convex	$\rightarrow$ Heuristics
Heuristic choice:	
Small neighbourhood, deterministic	$\rightarrow$ Steepest Descent (simple)
Can define tabu + full $N(x)$	$\rightarrow$ Tabu Search
Large $N(x)$, need randomness	$\rightarrow$ Simulated Annealing
Solution = "building blocks"	$\rightarrow$ Genetic Algorithm
Have domain knowledge	$\rightarrow$ Dedicated problem-specific heuristic

MILP Formulation Recipe

Step 1: Identify decision variables

- What are you deciding? (yes/no $\rightarrow$ binary, count $\rightarrow$ integer, amount $\rightarrow$ real)

- Name them clearly with their meaning and domain

Step 2: Write objective function

- What to maximise/minimise?
- Express as linear combination of decision variables

Step 3: Write constraints

- For each restriction: express as linear (in)equality
- Check: are all variables/indices in the right range?
- Capacity constraints: typically $\sum \text{resource_usage} \leq \text{limit}$
- Coverage constraints: typically $\sum \text{covers} \geq 1$
- Flow conservation: $\sum \text{in} = \sum \text{out} \pm \text{demand}$

Step 4: Specify variable types and bounds

- $x_i \in \{0,1\}$ for binary
- $x_i \in \mathbb{Z}$, $x_i \geq 0$ for non-negative integer
- $x_i \in \mathbb{R}$, $x_i \geq 0$ for non-negative real

Network Problem Classification

Is it a known graph problem?

- |— YES: Shortest path (one commodity, no capacity) → Dijkstra
- |— YES: Min-cost flow (one commodity, with capacity) → SSP algorithm
- |— YES: MST → Kruskal / Prim
- |— NO: Need optimisation techniques
 - |— Linear? → LP or (MI)LP
 - |— All continuous → LP → Simplex
 - |— Some integer → (MI)LP → Branch-and-Bound
 - |— Too complex / nonlinear → Heuristics
 - |— Deterministic, full $N(x)$: Tabu Search
 - |— Stochastic, partial $N(x)$: Simulated Annealing
 - |— Population-based: Genetic Algorithms

Study guide covers: Chapter 4 — MOBAN 2025-2026 (Prof. Mario Pickavet, Ghent University/IDLab)